

Project 5: Customer Segmentation with Clustering

Project Objective: To use unsupervised machine learning to identify distinct groups of customers within a mall's dataset. By segmenting customers based on multiple dimensions (income, age, spending habits), we can provide rich, actionable insights to the marketing team for highly targeted campaigns.

Core Concepts We'll Cover:

1. **Unsupervised Learning:** Understanding the difference between supervised and unsupervised learning.
2. **Clustering Fundamentals & K-Means:** A deep dive into how the K-Means algorithm works.
3. **The Elbow Method:** A critical technique for finding the optimal number of clusters (k).
4. **Multi-dimensional EDA:** Exploring relationships between all features using 2D and 3D plots.
5. **Multiple Segmentation Models:** Building and interpreting different clustering models based on different feature sets.
6. **Hierarchical Clustering:** A brief introduction to an alternative clustering method.
7. **Data-Driven Personas:** Translating cluster statistics into actionable business strategies.

Theoretical Concept: What is Unsupervised Learning?

So far, we have worked with **supervised learning**, where our datasets had a specific target variable we were trying to predict (e.g., `SalePrice`, `Survived`). The model *learns* the relationship between features and the known target.

Unsupervised learning is different. Here, we work with datasets that **do not have a target variable**. The goal is not to predict a known outcome, but to discover hidden patterns, structures, and relationships within the data itself.

Clustering is the most common type of unsupervised learning. The objective is to group data points into clusters, such that points within the same cluster are very similar to each other, and points in different clusters are very dissimilar.

Step 1: Setup - Importing Libraries and Loading Data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import scipy.cluster.hierarchy as sch

from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler

# Set plot style
sns.set_style('whitegrid')
```

Imagine you have a bunch of unsorted toys (your data points) and you want to group them into different boxes (your clusters) based on how similar they are. K-Means is like a helper that does this for you!

K-Means Algorithm: Grouping the Toys

- Choose how many boxes you want (Choose k): You decide beforehand how many groups you want to create. Let's say you want 3 boxes (so $k=3$).
- Place some "toy leaders" in the boxes (Initialize centroids): K-Means randomly picks 3 toys and places one in each box. These toys are the "leaders" or "centroids" of their boxes for now.
- Sort the toys by how close they are to the leaders (Assign points to clusters): K-Means looks at every single toy and figures out which "toy leader" it's closest to. It then puts each toy into the box with the closest leader.
- Find the new average location of the leaders (Update centroids): Once all the toys are in boxes, K-Means calculates the average location of all the toys in each box. These average locations become the new "toy leaders" or centroids.
- Repeat steps 3 and 4 until the leaders stop moving (Iterate until convergence): K-Means keeps repeating the sorting and leader-finding steps. It does this until the "toy leaders" don't move much anymore, meaning the groups are stable. Once the algorithm stops, you have your toys sorted into k boxes (clusters), where the toys in each box are generally similar to each other.

```
# Load the dataset from the user-provided file
!git clone "https://github.com/HarshvardhanSingh-13/Datasets"
df = pd.read_csv('/content/Datasets/Mall Dataset/Mall_Customers.csv')

print("Dataset loaded successfully.")
print(f>Data shape: {df.shape}")
df.head()
```

Cloning into 'Datasets'...

```
remote: Enumerating objects: 335, done.ote: Counting objects: 100%
(93/93), done.ote: Compressing objects: 100% (80/80), done.ote: Total
335 (delta 22), reused 0 (delta 0), pack-reused 242 (from 1)
```

```
{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 200,\n  \"fields\": [\n    {\n      \"column\": \"CustomerID\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 57,\n        \"min\": 1,\n        \"max\": 200,\n        \"num_unique_values\": 200,\n        \"samples\": [\n          96,\n          16,\n          31\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Gender\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n        \"samples\":
```

```
[
  {
    "semantic_type": "Female",
    "description": "Female",
    "column": "Age",
    "properties": {
      "dtype": "number",
      "std": 13,
      "min": 18,
      "max": 70,
      "num_unique_values": 51,
      "samples": [
        55, 26
      ]
    },
    "semantic_type": "Male",
    "description": "Male",
    "column": "Annual Income (k$)",
    "properties": {
      "dtype": "number",
      "std": 26,
      "min": 15,
      "max": 137,
      "num_unique_values": 64,
      "samples": [
        87, 101
      ]
    },
    "semantic_type": "Spending Score (1-100)",
    "description": "Spending Score (1-100)",
    "column": "Spending Score (1-100)",
    "properties": {
      "dtype": "number",
      "std": 25,
      "min": 1,
      "max": 99,
      "num_unique_values": 84,
      "samples": [
        39, 83
      ]
    },
    "semantic_type": "Spending Score (1-100)",
    "description": "Spending Score (1-100)",
    "column": "Spending Score (1-100)",
    "properties": {
      "dtype": "number",
      "std": 25,
      "min": 1,
      "max": 99,
      "num_unique_values": 84,
      "samples": [
        39, 83
      ]
    }
  },
  {
    "semantic_type": "Female",
    "description": "Female",
    "column": "Age",
    "properties": {
      "dtype": "number",
      "std": 13,
      "min": 18,
      "max": 70,
      "num_unique_values": 51,
      "samples": [
        55, 26
      ]
    },
    "semantic_type": "Male",
    "description": "Male",
    "column": "Annual Income (k$)",
    "properties": {
      "dtype": "number",
      "std": 26,
      "min": 15,
      "max": 137,
      "num_unique_values": 64,
      "samples": [
        87, 101
      ]
    },
    "semantic_type": "Spending Score (1-100)",
    "description": "Spending Score (1-100)",
    "column": "Spending Score (1-100)",
    "properties": {
      "dtype": "number",
      "std": 25,
      "min": 1,
      "max": 99,
      "num_unique_values": 84,
      "samples": [
        39, 83
      ]
    },
    "semantic_type": "Spending Score (1-100)",
    "description": "Spending Score (1-100)",
    "column": "Spending Score (1-100)",
    "properties": {
      "dtype": "number",
      "std": 25,
      "min": 1,
      "max": 99,
      "num_unique_values": 84,
      "samples": [
        39, 83
      ]
    }
  }
],
{
  "type": "dataframe",
  "variable_name": "df"
}
```

Step 2: In-Depth Exploratory Data Analysis (EDA)

Before clustering, we must understand the features and their relationships to find the most promising dimensions for segmentation.

```
# Basic info and statistics
print("Dataset Info:")
df.info()

# Drop the CustomerID as it's just an identifier and not useful for clustering
df.drop('CustomerID', axis=1, inplace=True)

print("\nDescriptive Statistics:")
print(df.describe())
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   CustomerID            200 non-null    int64
 1   Gender                200 non-null    object
 2   Age                  200 non-null    int64
 3   Annual Income (k$)    200 non-null    int64
 4   Spending Score (1-100) 200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

Descriptive Statistics:

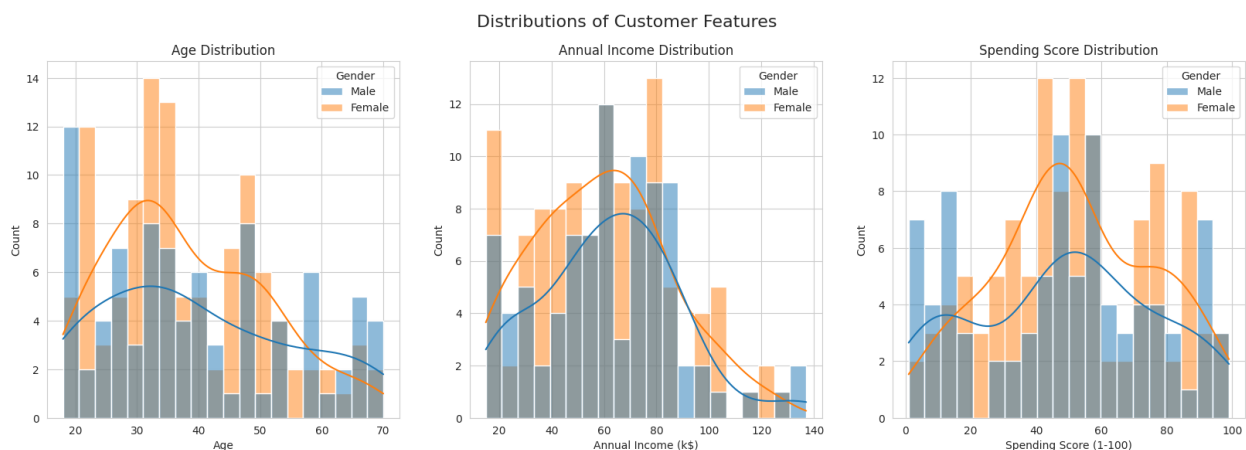
	Age	Annual Income (k\$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000
mean	38.850000	60.560000	50.200000
std	13.969007	26.264721	25.823522
min	18.000000	15.000000	1.000000
25%	28.750000	41.500000	34.750000
50%	36.000000	61.500000	50.000000
75%	49.000000	78.000000	73.000000
max	70.000000	137.000000	99.000000

2.1 Univariate Analysis (Feature Distributions)

```
fig, axes = plt.subplots(1, 3, figsize=(20, 6))
fig.suptitle('Distributions of Customer Features', fontsize=16)

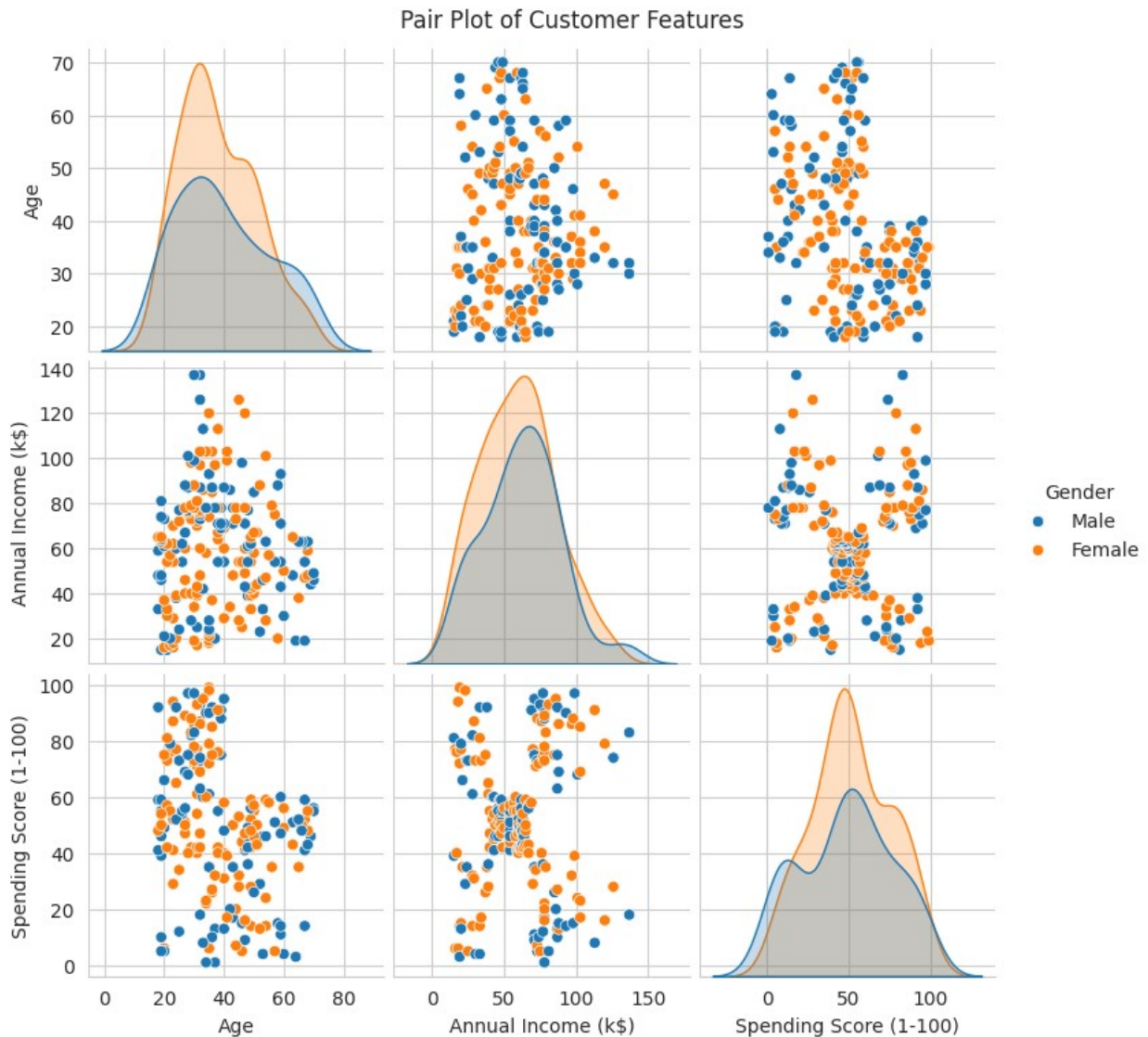
sns.histplot(ax=axes[0], data=df, x='Age', kde=True, bins=20,
             hue='Gender').set_title('Age Distribution')
sns.histplot(ax=axes[1], data=df, x='Annual Income (k$)', kde=True,
             bins=20, hue='Gender').set_title('Annual Income Distribution')
sns.histplot(ax=axes[2], data=df, x='Spending Score (1-100)',
             kde=True, bins=20, hue='Gender').set_title('Spending Score
Distribution')

plt.show()
```



2.2 Bivariate Analysis (Looking for Relationships)

```
# Using pairplot to visualize relationships between all numerical
variables
sns.pairplot(df, vars=['Age', 'Annual Income (k$)', 'Spending Score
(1-100)'], hue='Gender', diag_kind='kde')
plt.suptitle('Pair Plot of Customer Features', y=1.02)
plt.show()
```



CRITICAL INSIGHT from Pair Plot:

- The **Annual Income vs. Spending Score** plot is the most revealing. It clearly shows distinct, blob-like groups, making it the perfect candidate for clustering.
- The **Age vs. Spending Score** plot shows some patterns but they are less defined. We see a concentration of high-spending customers in the younger age brackets.
- **Gender** does not appear to be a strong separating factor in any of these plots, as the distributions for males and females largely overlap.

2.3 3D Visualization

Let's visualize the three most important features together in an interactive 3D plot to see if the clusters hold up.

```
fig = px.scatter_3d(df,
                    x='Annual Income (k$)',
```

```
y='Spending Score (1-100)',  
z='Age',  
color='Gender',  
title='3D View of Customer Data')  
  
fig.show()
```

Step 3: Segmentation Model 1 - Income & Spending Score

This is our primary segmentation model, focusing on the most promising features identified in the EDA.

Theoretical Concept: The Elbow Method

How do we choose the best number of clusters (k) for our K-Means model? The **Elbow Method** is a common technique.

It works by calculating the **Within-Cluster Sum of Squares (WCSS)** for different values of k (typically from 1 to 10). WCSS is the sum of the squared distances between each point and the centroid of the cluster it belongs to. A lower WCSS means the points are closer to their cluster centroids, indicating tighter clusters.

We then plot WCSS against the number of clusters (k). As k increases, WCSS will always decrease (because each point gets closer to its assigned centroid). However, at some point, adding more clusters doesn't significantly reduce the WCSS. This point, where the rate of decrease sharply changes, looks like an "elbow" in the plot.

The **optimal number of clusters (k)** is typically chosen at this elbow point. It represents a good balance between having too few clusters (high WCSS) and too many clusters (overfitting and diminishing returns).

Here are the most common init options available:

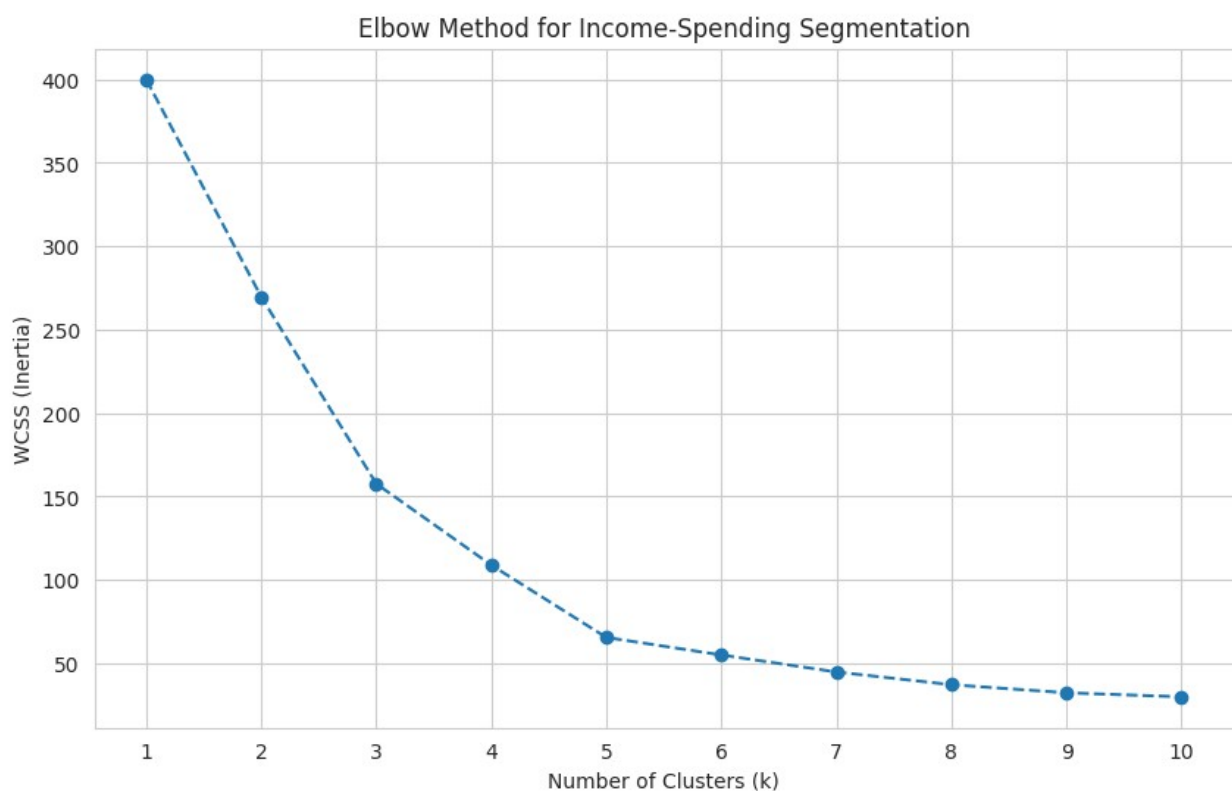
- 'k-means++' (Default): This is the recommended method. It smartly initializes the centroids to be spread out from each other. This significantly improves the chances of finding a globally optimal solution and reduces the risk of converging to a suboptimal local minimum.
- 'random': This method chooses $n_clusters$ observations (data points) randomly from the dataset and uses them as the initial centroids. While simple, it can sometimes lead to poor clustering results if the random initial centroids are not well-separated.
- A NumPy array: You can also provide a NumPy array with a shape of ($n_clusters$, $n_features$) that explicitly specifies the initial coordinates of the centroids. This is useful if you have prior knowledge about where the clusters should be located or if you want to replicate a specific initialization.

```
# 1. Select and scale the features  
X1 = df[['Annual Income (k$)', 'Spending Score (1-100)']]  
scaler1 = StandardScaler()  
X1_scaled = scaler1.fit_transform(X1)
```

```
# 2. Implement the Elbow Method
```

```
wcss1 = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42,
n_init=10)
    kmeans.fit(X1_scaled)
    wcss1.append(kmeans.inertia_)

# 3. Plot the Elbow Curve
plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), wcss1, marker='o', linestyle='--')
plt.title('Elbow Method for Income-Spending Segmentation')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('WCSS (Inertia)')
plt.xticks(range(1, 11))
plt.show()
```



Imagine you've grouped your friends into different clubs (these are your clusters). WCSS helps you figure out how "together" or "spread out" the members are within each club.

Cluster: This is one of the groups you've formed. Centroid: This is the "center point" of a cluster. Think of it as the average location of all the members in that club. WCSS: For each cluster, you calculate the distance from every member in that cluster to the cluster's centroid. Then you square all these distances and add them up. You do this for all the clusters and add up the totals. A low WCSS means that the points within each cluster are very close to their centroid. This suggests that the clusters are tight and well-defined.

A high WCSS means that the points within each cluster are more spread out from their centroid, indicating less distinct or looser clusters.

How WCSS relates to K-Means:

In K-Means, the algorithm's goal is to minimize the WCSS. It tries to find the best locations for the centroids and the best groupings of points so that the total WCSS is as small as possible.

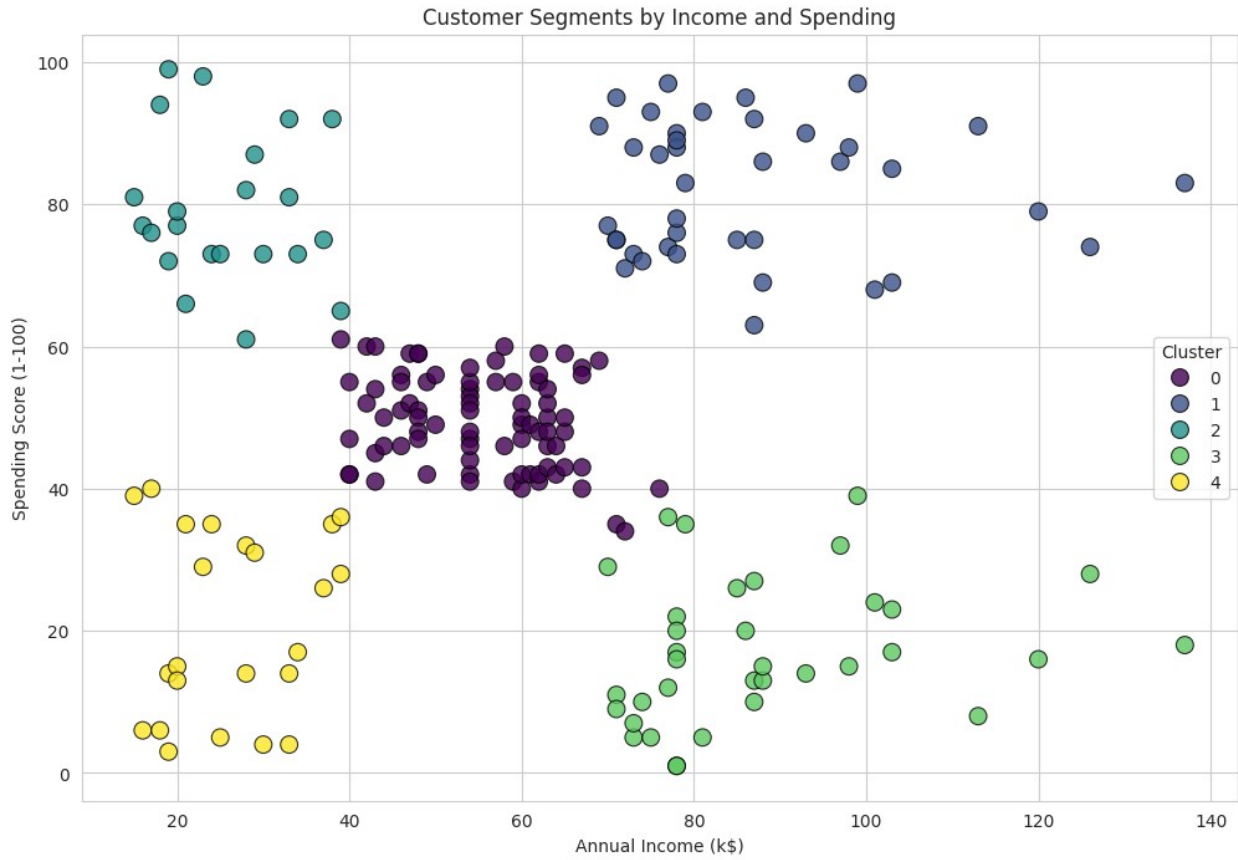
The Elbow Method, which you saw in the notebook, uses WCSS to help choose the optimal number of clusters (k) for K-Means. You calculate the WCSS for different values of k and look for the "elbow" where the decrease in WCSS starts to slow down. This point suggests that adding more clusters doesn't significantly improve the "tightness" of the clusters.

Observation: The "elbow" of the curve is clearly at **k=5**. The WCSS decreases sharply until k=5, and then the rate of decrease flattens out. This confirms our visual intuition from the scatter plot!

```
# Build and fit the final model
kmeans1 = KMeans(n_clusters=5, init='k-means++', random_state=42,
n_init=10)
df['Income_Cluster'] = kmeans1.fit_predict(X1_scaled)
```

3.1 Visualizing and Interpreting the Income-Based Segments

```
plt.figure(figsize=(12, 8))
sns.scatterplot(data=df, x='Annual Income (k$)', y='Spending Score (1-100)',
hue='Income_Cluster', palette='viridis', s=100,
alpha=0.8, edgecolor='black')
plt.title('Customer Segments by Income and Spending')
plt.legend(title='Cluster')
plt.show()
```

Quantitative Persona Analysis

```
cluster_profiles1 = df.groupby('Income_Cluster')[['Age', 'Annual
Income (k$)', 'Spending Score (1-100)']].mean().round(2)
cluster_profiles1['Size'] = df['Income_Cluster'].value_counts()
print("--- Income-Based Cluster Profiles ---")
cluster_profiles1
```

--- Income-Based Cluster Profiles ---

```
{"summary":{"name": "cluster_profiles1", "rows": 5,
"fields": [{"column": "Income_Cluster",
"properties": {"dtype": "int32",
"num_unique_values": 5,
"samples": [1, 4, 2],
"semantic_type": "",
"description": ""}],
{"column": "Age",
"properties": {"dtype": "number",
"std": 8.254390952699055,
"min": 25.27,
"max": 45.22,
"num_unique_values": 5,
"samples": [32.69, 45.22, 25.27],
"semantic_type": "",
"description": ""}],
{"column": "Annual Income (k$)",
"properties": {"dtype": "number",
"std": 30.6900941673368,
"min": 25.73,
"max": 88.2,
"num_unique_values": 5,
"samples": [86.54, ]}}
```

```

26.3,\n          25.73\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          },\n          {\n          \"column\":\n          \"Spending Score (1-100)\",\n          \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 30.91265485201813,\n          \"min\": 17.11,\n          \"max\": 82.13,\n          \"num_unique_values\": 5,\n          \"samples\": [\n          82.13,\n          20.91,\n          79.36\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          },\n          {\n          \"column\": \"Size\",\n          \"properties\": {\n          \"dtype\":\n          \"number\",\n          \"std\": 24,\n          \"min\": 22,\n          \"max\": 81,\n          \"num_unique_values\": 5,\n          \"samples\":\n          [\n          39,\n          23,\n          22\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          }\n          }\n          ]\n          }],\n          \"type\": \"dataframe\", \"variable_name\": \"cluster_profiles1\"}

```

3.2 Income-Based Personas and Strategy

Using the quantitative profiles, we can define our segments:

- **Cluster 0 (Careful High-Earners):** High income (~88k) but low spending (~17). Cautious, value-driven buyers.
- **Cluster 1 (Standard):** Average income (~55k) and spending (~50). This is the largest, core customer group.
- **Cluster 2 (Budget Shoppers):** Low income (~26k) and low spending (~21). Highly price-sensitive.
- **Cluster 3 (Target - High Value):** High income (~87k) and high spending (~82). The mall's prime target.
- **Cluster 4 (Enthusiasts):** Low income (~26k) but high spending (~79). Likely young, trend-focused shoppers.

Step 4: Segmentation Model 2 - Age & Spending Score

Let's create a different segmentation model to see if we can find age-related customer groups.

```

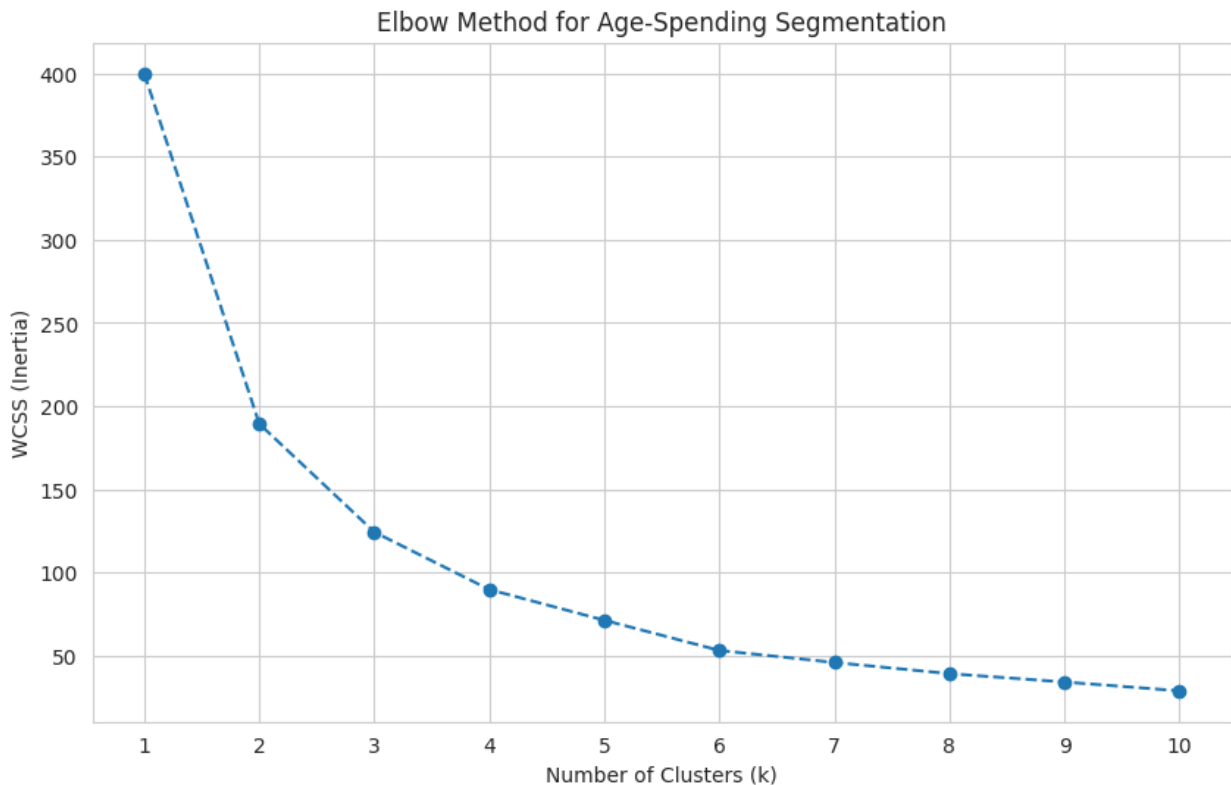
# 1. Select and scale the features
X2 = df[['Age', 'Spending Score (1-100)']]
scaler2 = StandardScaler()
X2_scaled = scaler2.fit_transform(X2)

# 2. Implement the Elbow Method
wcss2 = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42,
n_init=10)
    kmeans.fit(X2_scaled)
    wcss2.append(kmeans.inertia_)

# 3. Plot the Elbow Curve

```

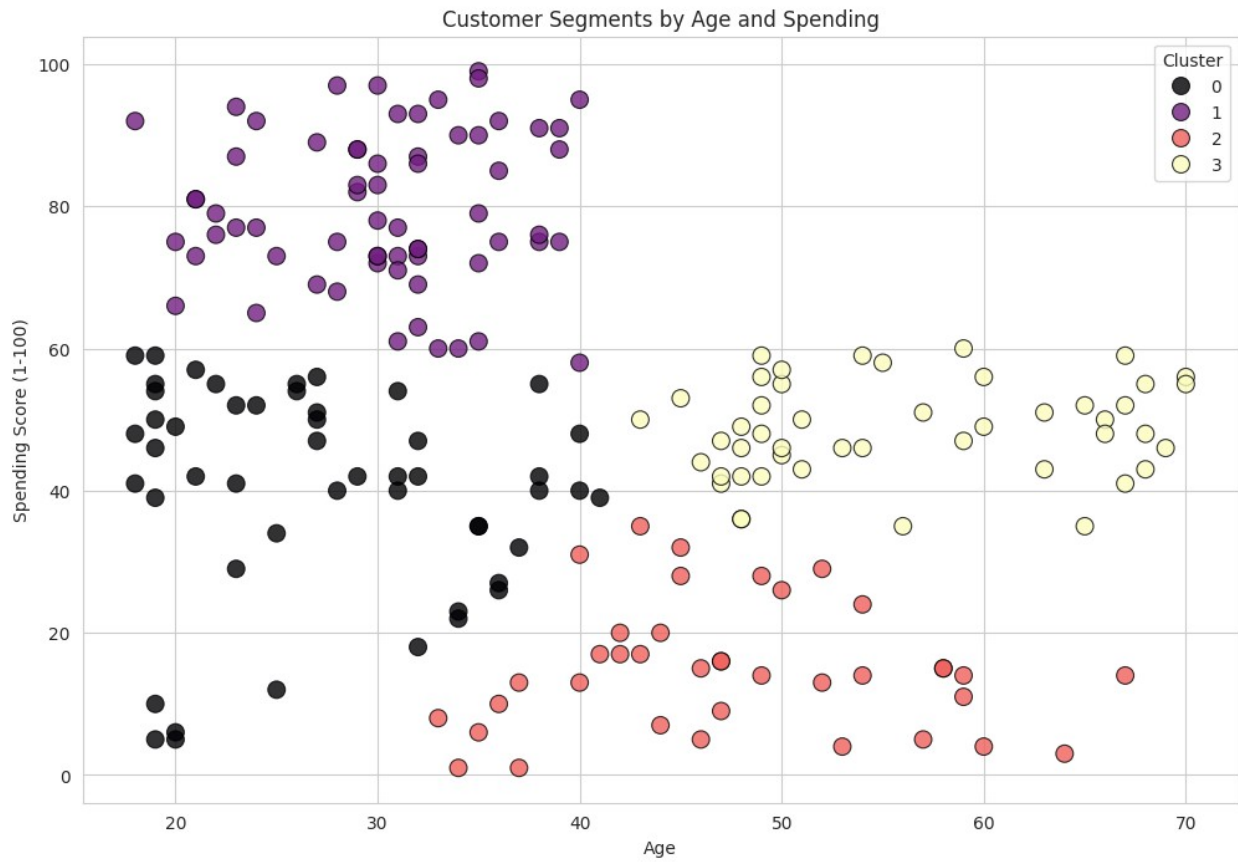
```
plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), wcss2, marker='o', linestyle='--')
plt.title('Elbow Method for Age-Spending Segmentation')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('WCSS (Inertia)')
plt.xticks(range(1, 11))
plt.show()
```



Observation: The elbow is less sharp here, but **k=4** appears to be a reasonable choice.

```
# Build and fit the final model for age segmentation
kmeans2 = KMeans(n_clusters=4, init='k-means++', random_state=42,
n_init=10)
df['Age_Cluster'] = kmeans2.fit_predict(X2_scaled)

# Visualize the new clusters
plt.figure(figsize=(12, 8))
sns.scatterplot(data=df, x='Age', y='Spending Score (1-100)',
hue='Age_Cluster', palette='magma', s=100, alpha=0.8,
edgecolor='black')
plt.title('Customer Segments by Age and Spending')
plt.legend(title='Cluster')
plt.show()
```



4.1 Interpreting the Age-Based Segments

This model gives us a different perspective:

- **Young High-Spenders:** A clear group of customers under 40 with high spending scores.
- **Young Low-Spenders:** Customers under 40 with low-to-moderate spending.
- **Older Average-Spenders:** Customers over 40 with moderate spending scores.
- **Older Low-Spenders:** A small group of older customers with very low spending scores.

Step 5: An Alternative Method - Hierarchical Clustering

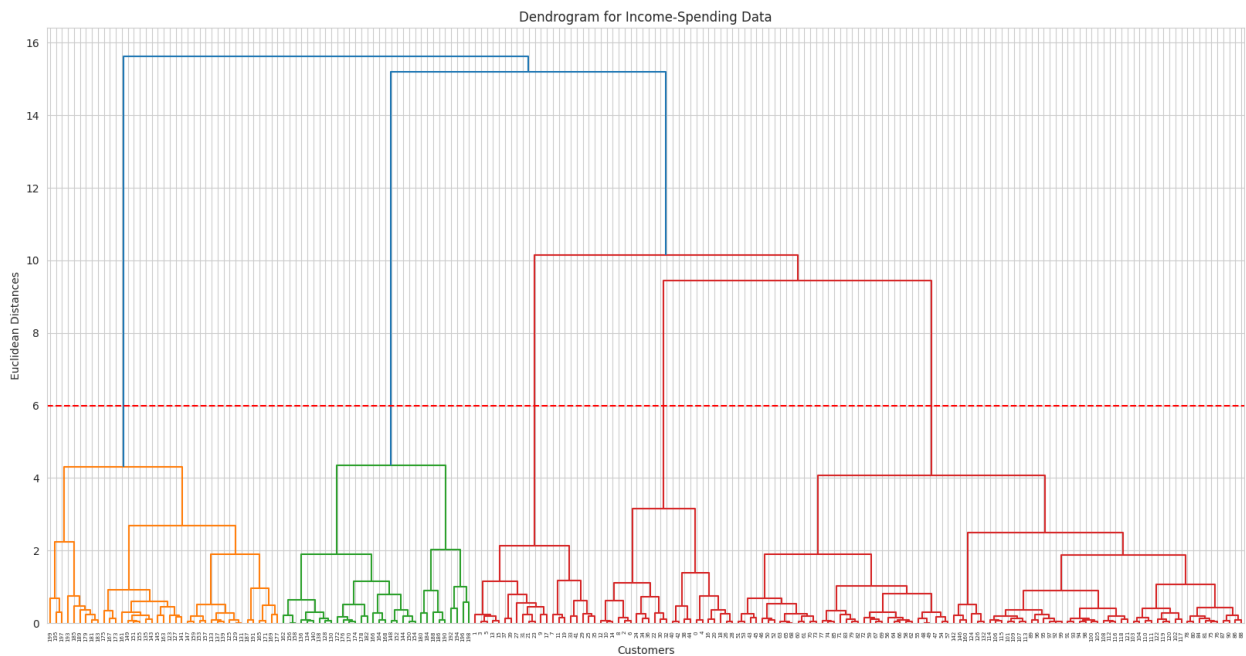
To add more depth, let's briefly look at another clustering technique.

Theoretical Concept: Hierarchical Clustering & Dendrograms

Unlike K-Means, Hierarchical Clustering doesn't require us to specify the number of clusters beforehand. It builds a tree-like hierarchy of clusters.

We can visualize this hierarchy with a **dendrogram**. The dendrogram shows how clusters are merged (or split). To find the optimal number of clusters, we look for the tallest vertical line that doesn't cross any horizontal lines and draw a horizontal line through it. The number of vertical lines our new line crosses is the optimal number of clusters.

```
plt.figure(figsize=(20, 10))
dendrogram = sch.dendrogram(sch.linkage(X1_scaled, method='ward'))
plt.title('Dendrogram for Income-Spending Data')
plt.xlabel('Customers')
plt.ylabel('Euclidean Distances')
plt.axhline(y=6, color='r', linestyle='--') # Example cut-off line
plt.show()
```



Insight: The dendrogram also confirms that **5 clusters** is the optimal number for the income-spending data. If we draw a horizontal line at a height of 6 (as shown by the red dashed line), it crosses five vertical lines.

Step 6: Conclusion

In this extended project, we successfully applied unsupervised machine learning to perform customer segmentation.

Key Steps Undertaken:

1. **Performed an in-depth EDA:** Used 2D and 3D plots to explore customer data from multiple angles.
2. **Built two distinct segmentation models:** Created an income-based model and an age-based model, showing that segmentation is not a one-size-fits-all process.
3. **Used the Elbow Method** to programmatically determine the optimal number of clusters for each scenario.
4. **Introduced Hierarchical Clustering** as an alternative method and used its dendrogram to validate our choice of k.
5. **Created quantitative, data-driven personas** for each segment, providing the marketing team with precise, actionable insights for targeted campaigns.

This project highlights how different clustering approaches can uncover different facets of customer behavior, providing a richer, more nuanced understanding than a single model alone.

Submission Criteria

Based on the project objectives and the insights gained from the initial analysis, here are the key criteria for your submission:

1. **Explore Gender vs. Spending Score:** Analyze the relationship between 'Gender' and 'Spending Score (1-100)' to see if gender plays a significant role in spending habits. Visualizations and summary statistics would be helpful here.
2. **Apply Feature Engineering for Clustering:** Create a new feature using existing data (Feature Engineering). Then, explore if clustering can be effectively performed using this newly engineered feature in combination with one or more existing features. Document the feature engineering process, the chosen features for clustering, the determination of the optimal number of clusters (e.g., using the Elbow Method), and the resulting cluster visualization and interpretation.